

CCM101 - Cloud Computing Enterprise Cloud Architect Operational Manual

Final Requirement

Project: Secure Multi-Tier Web Application (VaultCloud)

Application Stack: Nextcloud + MariaDB on Docker
Compose

Platform: Ubuntu Server 26.04.1 LTS on Oracle VirtualBox

Group: The Soft Lockups (We survived the VM freeze of 2026!)

Prepared by:

Bucac, Bryxtre

Malona, Kate

Meim, Ronald Joel

Reofrir, Jemma

Palma, JohnLester

BSIT – 4A

Instructor: Jenkielyn C. Torres

Date: October 2026

CONTENTS

1. PROJECT OVERVIEW	3
2. ARCHITECTURE AND INFRASTRUCTURE	5
3. VIRTUAL MACHINE SETUP	8
4. DOCKER AND COMPOSE DEPLOYMENT	12
5. PERSISTENT STORAGE	18
6. SECURITY AND FIREWALL	21
7. AUTOMATION AND SCHEDULED BACKUP	24
8. OPERATIONAL PROCEDURES	28
9. MAINTENANCE AND TROUBLESHOOTING	30
10. SOURCE CONTROL (GITHUB)	32
11. FINAL REFLECTION	34

List of Figures

Figure 1. Enterprise Cloud Infrastructure Architecture	5
Figure 2. VirtualBox Port Forwarding Rules	7
Figure 3. Ubuntu Server Virtual Machine Information	11
Figure 4. Docker Engine Verification (hello-world)	13
Figure 5. Docker Compose Container Status	16
Figure 6. Nextcloud Login Page in the Host Browser	16
Figure 7. Nextcloud Dashboard After Login	17
Figure 8. Docker Volumes and Stored Data	19
Figure 9. Containers Running After a VM Reboot	19
Figure 10. Test File Still Available After the Restart	20
Figure 11. UFW Firewall Status	22
Figure 12. Listening Ports and Database Isolation	23
Figure 13. Automation Script Permissions and Execution	25
Figure 14. Backup Integrity Verification	26
Figure 15. Scheduled Database Backup Using Cron	27

1. PROJECT OVERVIEW

1.1 Project Description

VaultCloud is a private, self-hosted file-sharing cloud built for the CCM101 final mission, The Enterprise Cloud Architect. It runs **Nextcloud** (application tier) and **MariaDB** (data tier) as two Docker containers managed by Docker Compose, on an **Ubuntu Server** virtual machine hosted in **Oracle VirtualBox** on a Windows laptop. It is persistent, secured and self-maintaining.

1.2 Objectives

1. Build a Linux virtual machine and document its architecture.
2. Deploy a multi-tier application using Docker Compose as infrastructure as code.
3. Keep application and database data safe across container re-creation and VM reboots.
4. Harden the server with a default-deny firewall and by never exposing the database.
5. Automate verified database backups with Bash and Cron, and document everything so another administrator can operate and recover the system.

1.3 Technology Stack

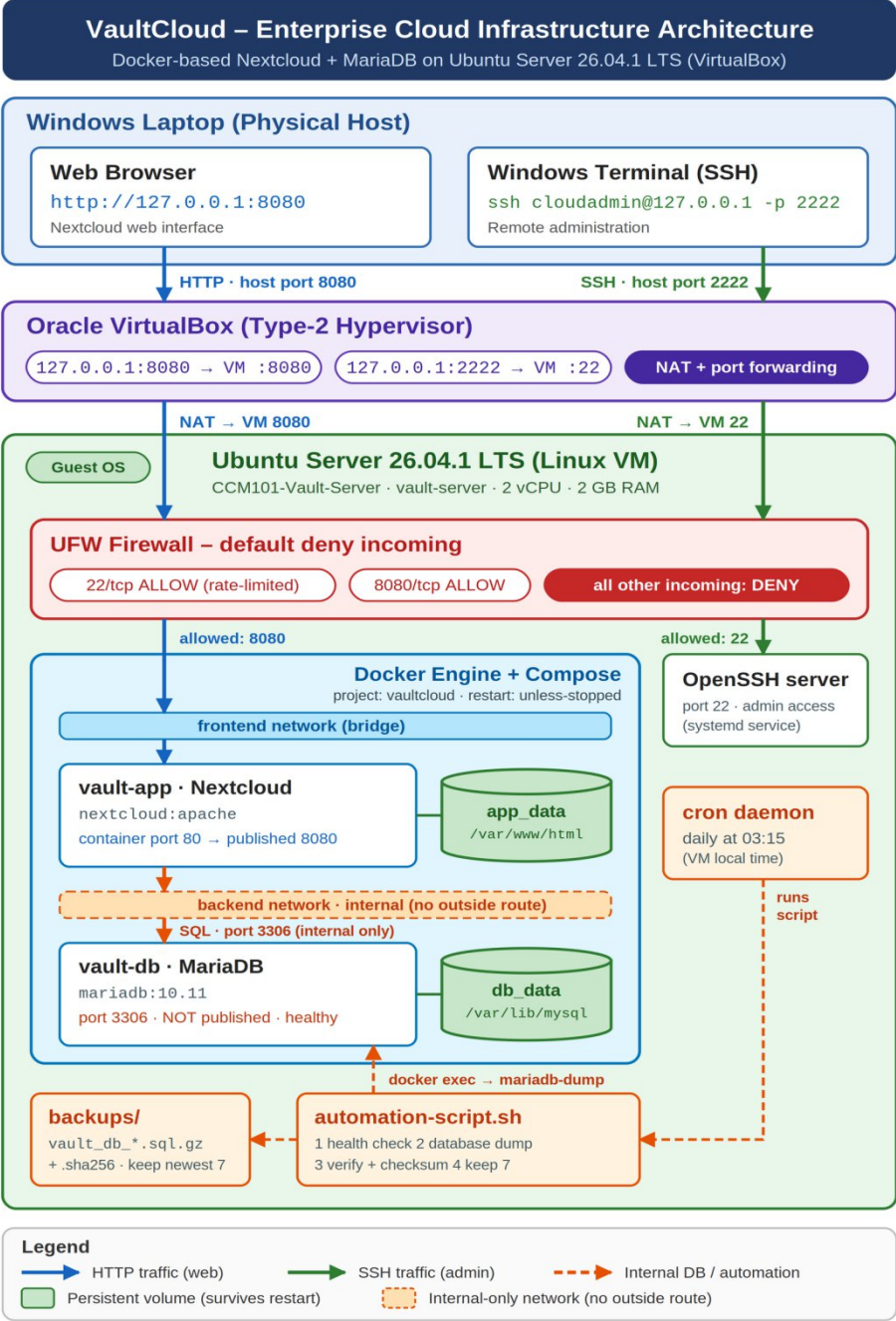
Layer	Technology	Role
Physical host	Windows laptop	Runs the hypervisor, browser and SSH client
Hypervisor	Oracle VirtualBox	Runs the Linux VM with NAT and port forwarding
Operating system	Ubuntu Server 26.04.1 LTS	Linux server inside the VM
Container platform	Docker Engine + Docker Compose	Runs and orchestrates the containers

Layer	Technology	Role
Application	Nextcloud (nextcloud:apache)	Web application for files and collaboration
Database	MariaDB 10.11 (mariadb:10.11)	Stores Nextcloud data
Firewall	UFW (Uncomplicated Firewall)	Default-deny protection of the VM
Automation	Bash script + Cron	Daily health check and database backup

1.4 Requirements Coverage

Requirement	How it is met	Section
1. Base infrastructure	Windows host, VirtualBox NAT, Ubuntu VM, diagram	2, 3
2. Infrastructure integration	Compose stack, health check, internal network, volumes	4, 5
3. Security configurations	UFW default deny; database not published	6
4. Automation scripts	automation-script.sh scheduled daily by Cron	7
5. Documentation	This manual, README, reflection, GitHub	8–11

2. ARCHITECTURE AND INFRASTRUCTURE



2.1 Infrastructure Configuration

Item	Value
Physical host	Windows laptop
Hypervisor	Oracle VirtualBox (NAT networking)
Virtual machine name	CCM101-Vault-Server
Operating system	Ubuntu Server 26.04.1 LTS (kernel 7.0.0-38-generic, x86-64)
Hostname / admin user	vault-server / cloudadmin
CPU	2 virtual CPUs
Memory	About 2 GB RAM (1.6 GiB visible inside the VM)
Storage	25 GB virtual disk
VM network	NAT, guest address 10.0.2.15 (interface enp0s3)
Web access from host	http://127.0.0.1:8080
SSH access from host	127.0.0.1 port 2222

2.2 Network Design

The VM uses **NAT** networking, so it is isolated from the physical network. VirtualBox forwards two ports from the host loopback address to the VM. Nothing else on the VM can be reached from the host, which keeps the exposed surface small.

Rule	Host (Windows)	Guest (Ubuntu VM)	Purpose
Web	127.0.0.1 : 8080	port 8080	Nextcloud website
SSH	127.0.0.1 : 2222	port 22	Remote administration

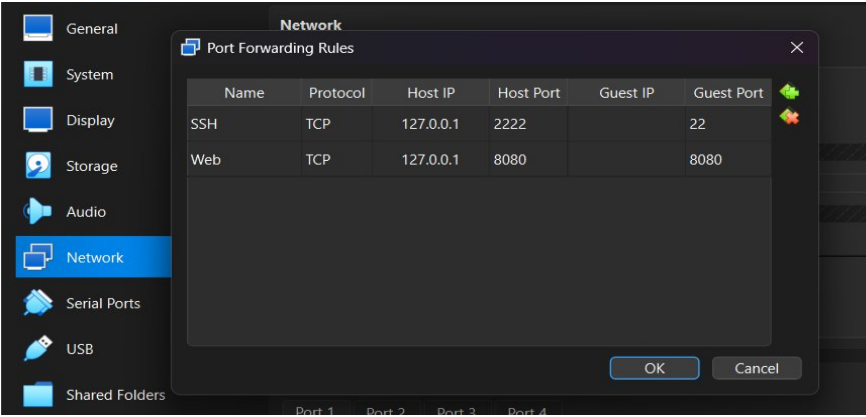


Figure 2. VirtualBox Port Forwarding Rules

What it shows:

The two NAT rules of the CCM101-Vault-Server VM: **SSH** (TCP, host 127.0.0.1:2222 to guest port 22) and **Web** (TCP, host 127.0.0.1:8080 to guest port 8080).

3. VIRTUAL MACHINE SETUP

This section describes how the Ubuntu Server virtual machine is created, installed and verified.

3.1 Create the Virtual Machine in VirtualBox

1. Open **Oracle VirtualBox** and click **New**.
2. Enter the name CCM101-Vault-Server, choose the Ubuntu Server ISO, set Type **Linux** and Version **Ubuntu (64-bit)**.
3. Tick **Skip Unattended Installation** so the installer can be run manually.
4. Set the hardware: **2048 MB** memory and **2 processors**.
5. Create a virtual hard disk (dynamically allocated) and click **Finish**.

Setting	Value
Name	CCM101-Vault-Server
Type / Version	Linux / Ubuntu (64-bit)
Base memory	2048 MB (the laptop has limited RAM, so the VM is kept small)
Processors	2
Virtual disk	25 GB, dynamically allocated
Network adapter 1	NAT, with the port-forwarding rules in Section 2.2

3.2 Configure Port Forwarding

1. Select the VM, click **Settings**, then **Network**, then the **Adapter 1** tab.
 2. Confirm **Attached to: NAT**, expand **Advanced**, and click **Port Forwarding**.
 3. Click the green **+** button and add the two rules shown in Figure 2 (SSH 2222 to 22, Web 8080 to 8080).
-

- 4. Click **OK** twice to save.

3.3 Install Ubuntu Server

Start the VM. The installer is text-based, so use the arrow keys, **Enter**, **Space** and **Tab**. Click inside the VM window to type; press the **Right Ctrl** key to release the mouse.

Installer screen	Choice
Language / keyboard	English / layout that matches the physical keyboard
Type of install	Ubuntu Server (not minimized)
Network	Default DHCP address (10.0.2.15 on enp0s3)
Storage	Use an entire disk (LVM option unticked)
Profile setup	Server name vault-server, username cloudadmin, strong password
SSH setup	Tick Install OpenSSH server
Featured snaps	None (Docker is installed later from Docker's repository)
Finish	Reboot now, then press Enter if asked to remove the installation medium

3.4 First Configuration

Log in at the VM console and update the system:

```
sudo apt update && sudo apt full-upgrade -y
sudo timedatectl set-timezone Asia/Manila
ip -br a
```

Setting the time zone makes the 3:15 AM backup run at 3:15 AM local time.

3.5 Connect over SSH from Windows

Use SSH from Windows Terminal instead of the VM console. It supports copy and paste and avoids keyboard-layout problems in the VirtualBox console.

```
ssh cloudadmin@127.0.0.1 -p 2222
```

Type yes at the first fingerprint prompt, then enter the password.

3.6 Verify the Virtual Machine

```
hostnamectl  
free -h  
nproc  
lsb_release -a
```

Figure 3. Ubuntu Server Virtual Machine Information

```

cloudadmin@vault-server: ~  X  +  v
0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

cloudadmin@vault-server:~$ hostnamectl
Static hostname: vault-server
    Icon name: computer-vm
    Chassis: vm
    Machine ID: 083fa1ba19df4b1aa52969383e6d798a
    Boot ID: e4e1016f116144a996cc245931f9eb7e
Virtualization: oracle
Operating System: Ubuntu 26.04.1 LTS
    Kernel: Linux 7.0.0-38-generic
    Architecture: x86-64
    Hardware Vendor: innotek GmbH
    Hardware Model: VirtualBox
Hardware Version: 1.2
Firmware Version: VirtualBox
    Firmware Date: Fri 2006-12-01
    Firmware Age: 19y 10month 6d
cloudadmin@vault-server:~$ free -h
               total        used        free      shared    buff/cache   available
Mem:           1.6Gi         399Mi         557Mi         1.1Mi         854Mi         1.2Gi
Swap:          2.0Gi           0B          2.0Gi
cloudadmin@vault-server:~$ nproc
2
cloudadmin@vault-server:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 26.04.1 LTS
Release:        26.04
Codename:       resolute
cloudadmin@vault-server:~$ |

```

What it shows:

hostnamectl shows hostname **vault-server**, **Ubuntu 26.04.1 LTS**, kernel **Linux 7.0.0-38-generic**, architecture **x86-64** and virtualization **oracle** (VirtualBox). free -h shows about 1.6 GiB of memory, nproc shows **2** CPUs and lsb_release -a confirms release 26.04.

4. DOCKER AND COMPOSE DEPLOYMENT

4.1 Install Docker Engine and Compose

Docker is installed from Docker's official apt repository. Run the blocks below in order.

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL
https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

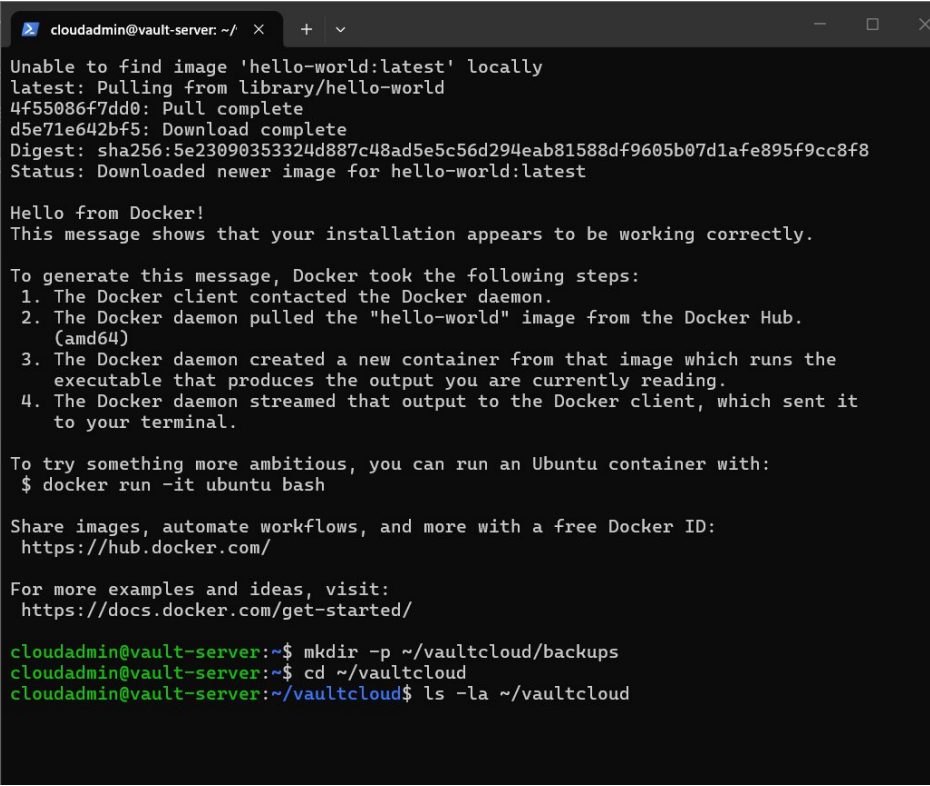
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-
$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt update

sudo apt install -y docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER
```

Log out and connect again so the new group applies, then verify:

```
docker --version
docker compose version
sudo systemctl is-enabled docker
docker run --rm hello-world
```

Figure 4. Docker Engine Verification (hello-world)



```

cloudadmin@vault-server: ~/  x  +  v
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

cloudadmin@vault-server:~$ mkdir -p ~/vaultcloud/backups
cloudadmin@vault-server:~$ cd ~/vaultcloud
cloudadmin@vault-server:~/vaultcloud$ ls -la ~/vaultcloud

```

What it shows:

Running `docker run --rm hello-world` pulls the **hello-world** image from Docker Hub and prints **Hello from Docker!**, which confirms that Docker Engine is installed and that the Docker client, the Docker daemon and image downloads all work. The Compose plugin is confirmed in Figure 5, where docker compose manages the running containers. The commands at the bottom (`mkdir`, `cd`, `ls -la`) are the start of the next step, creating the project folder.

4.2 Project Files

"All files live in one project folder (`~/vaultcloud`) and in the GitHub repository. The complete code for every file

listed below is committed to the Laboratory-10-Enterprise-Cloud-Architect folder on GitHub."

File	Purpose
docker-compose.yml	Defines the Nextcloud and MariaDB services, networks and volumes
.env.example	Template for passwords and settings (copied to .env)
.env	Real passwords; never committed to GitHub (mode 600)
.gitignore	Keeps .env and backups/ out of GitHub
automation-script.sh	Health check and verified database backup
backups/	Folder holding the compressed backups and their checksums

4.3 How docker-compose.yml Works

Setting	Purpose
image: mariadb:10.11	Long-term-support database version supported by Nextcloud
healthcheck (db)	Marks the database healthy only when it accepts connections
depends_on: service_healthy	Nextcloud starts only after the database is healthy
networks: frontend + backend	Web traffic uses frontend; database traffic uses backend
backend: internal: true	The database network has no route to or from the outside world
ports: "8080:80" (app only)	The single published entry point; MariaDB publishes no port

Setting	Purpose
volumes: app_data, db_data	Application and database data are stored outside the containers
restart: unless-stopped	Containers start again after a crash or VM reboot
\${VARIABLES} from .env	No passwords are written in the compose file

4.4 Configure Secrets and Start the Stack

```
mkdir -p ~/vaultcloud/backups && cd ~/vaultcloud
cp .env.example .env
nano .env
chmod 600 .env
```

Change **every** change-me value in .env to a strong, different password. Save with **Ctrl+O**, **Enter**, **Ctrl+X**. Then validate and start:

```
docker compose config -q && echo "compose file OK"
docker compose up -d
docker compose ps
```

The first start downloads the images and lets Nextcloud install itself, which can take a few minutes.

4.5 Verify the Deployment

```
cloudadmin@vault-server:~$ cd /home/cloudadmin/vaultcloud
cloudadmin@vault-server:~/vaultcloud$ sudo docker compose up -d
[sudo: authenticate] Password:
[+] up 2/2
  V Container vault-db Healthy
  V Container vault-app Running
cloudadmin@vault-server:~/vaultcloud$ sudo docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
vault-app	nextcloud/apache	"/entrypoint.sh apac_..."	app	37 minutes ago	Up About a minute	0.0.0.0:8080->80/tcp, [::]:8080
vault-db	mariadb:10.11	"docker-entrypoint.s..."	db	37 minutes ago	Up About a minute (healthy)	3306/tcp

```
cloudadmin@vault-server:~/vaultcloud$
```

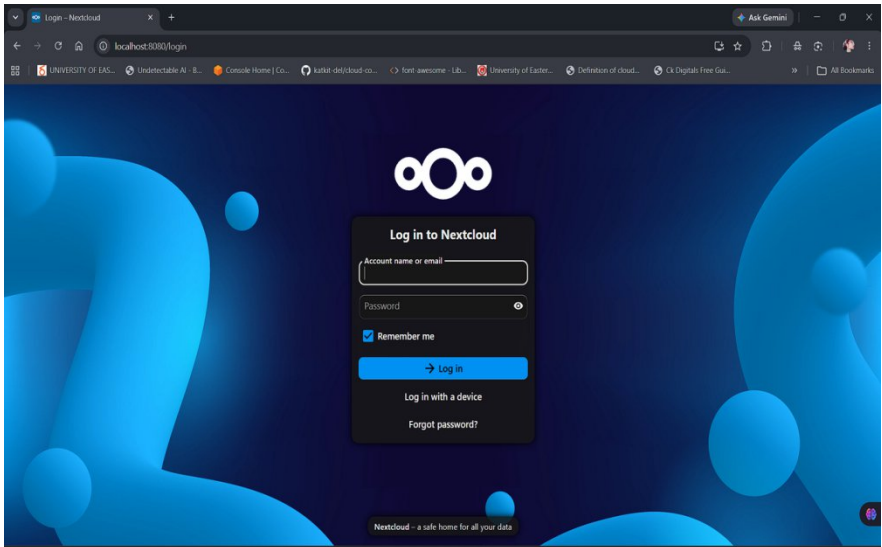
Figure 5. Docker Compose Container Status

What it shows:

Figure 7. Nextcloud Dashboard After Login

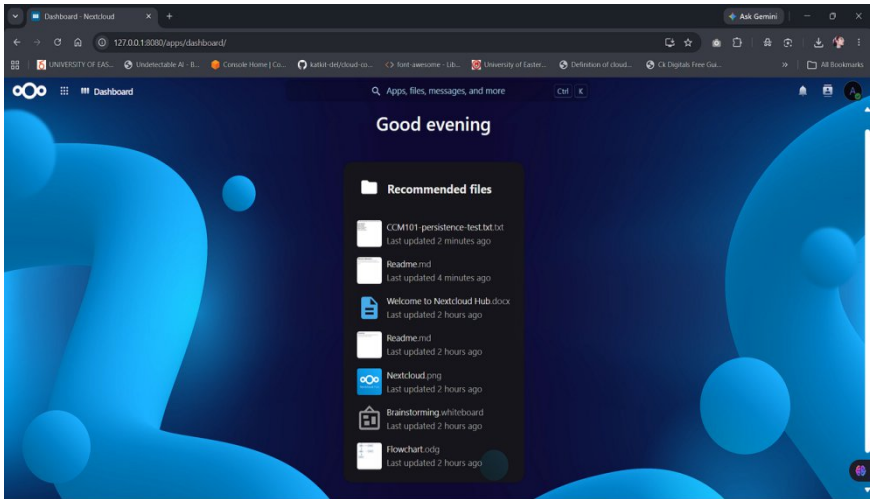
docker compose up -d reports both containers started. docker compose ps shows **vault-app** (nextcloud:apache) running with port 8080 mapped to container port 80 and **vault-db** (mariadb:10.11) running and **healthy** with only 3306/tcp listed, which means the database port is not published to the VM.

Figure 6. Nextcloud Login Page in the Host Browser



What it shows:

The Nextcloud login page is served at localhost:8080 in the Windows browser through the VirtualBox port-forwarding rule.



What it shows:

After signing in, the Nextcloud dashboard loads at 127.0.0.1:8080. The uploaded test file **CCM101-persistence-test.txt** appears under Recommended files.

5. PERSISTENT STORAGE

5.1 Storage Configuration

Containers are disposable, so everything that must be kept lives in named Docker volumes.

Volume	Mounted in container	Holds
vaultcloud_app_data	vault-app: /var/www/html	Nextcloud code, configuration and user files
vaultcloud_db_data	vault-db: /var/lib/mysql	MariaDB database files

5.2 Persistence Test Procedure

1. Sign in to Nextcloud and upload a small file named CCM101-persistence-test.txt.
2. Confirm the file exists inside the volume (Figure 8).
3. Restart the whole VM with `sudo reboot`, reconnect over SSH and run `uptime -p` and `docker compose ps` (Figure 9).
4. Refresh the browser and confirm the file is still listed (Figure 10).

Important: Never run `docker compose down -v` in normal operation. The `-v` option deletes the volumes and destroys all data.

5.3 Evidence

Figure 8. Docker Volumes and Stored Data

```

cloudadmin@vault-server:~/vaultcloud$ sudo docker[ 606.400591] watchdog: BUG: soft lockup - CPU#0 stuck for 23s! [swapper/0:0]
volume ls
DRIVER    VOLUME NAME
local     vaultcloud_app_data
local     vaultcloud_db_data
cloudadmin@vault-server:~/vaultcloud$ sudo docker exec vault-app ls -l /var/www/html/data/architect/files
total 34608
-rw-r--r-- 1 www-data www-data    171 Oct  7 11:11 CCM101-persistence-test.txt.txt
drwxr-xr-x 2 www-data www-data   4096 Oct  7 09:39 Documents
-rw-r--r-- 1 www-data www-data 30412936 Oct  7 09:39 Nextcloud Manual.pdf
-rw-r--r-- 1 www-data www-data 39630936 Oct  7 09:39 Nextcloud Intro.mp4
-rw-r--r-- 1 www-data www-data  50598 Oct  7 09:39 Nextcloud.png
drwxr-xr-x 2 www-data www-data   4096 Oct  7 09:39 Photos
-rw-r--r-- 1 www-data www-data    196 Oct  7 11:09 Readme.md
-rw-r--r-- 1 www-data www-data 976625 Oct  7 09:39 Reasons to use Nextcloud.pdf
drwxr-xr-x 2 www-data www-data   4096 Oct  7 09:39 Templates
-rw-r--r-- 1 www-data www-data   3168 Oct  7 09:39 Templates credits.md
cloudadmin@vault-server:~/vaultcloud$

```

What it shows:

docker volume ls lists the two persistent volumes **vaultcloud_app_data** and **vaultcloud_db_data**. Listing the Nextcloud data folder inside the container shows the uploaded **CCM101-persistence-test.txt** stored on the volume, together with the other Nextcloud files.

Figure 9. Containers Running After a VM Reboot

```

cloudadmin@vault-server:~$ cd /home/cloudadmin/vaultcloud
cloudadmin@vault-server:~/vaultcloud$ uptime -p
up 3 minutes
cloudadmin@vault-server:~/vaultcloud$ sudo docker compose ps
[sudo: authenticate] Password:

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
vault-app	nextcloud:apache	"/entrypoint.sh apac..."	app	9 minutes ago	Up 6 minutes	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
vault-db	mysql:5.7	"docker-entrypoint.s..."	db	9 minutes ago	Up 6 minutes (healthy)	3306/tcp

```

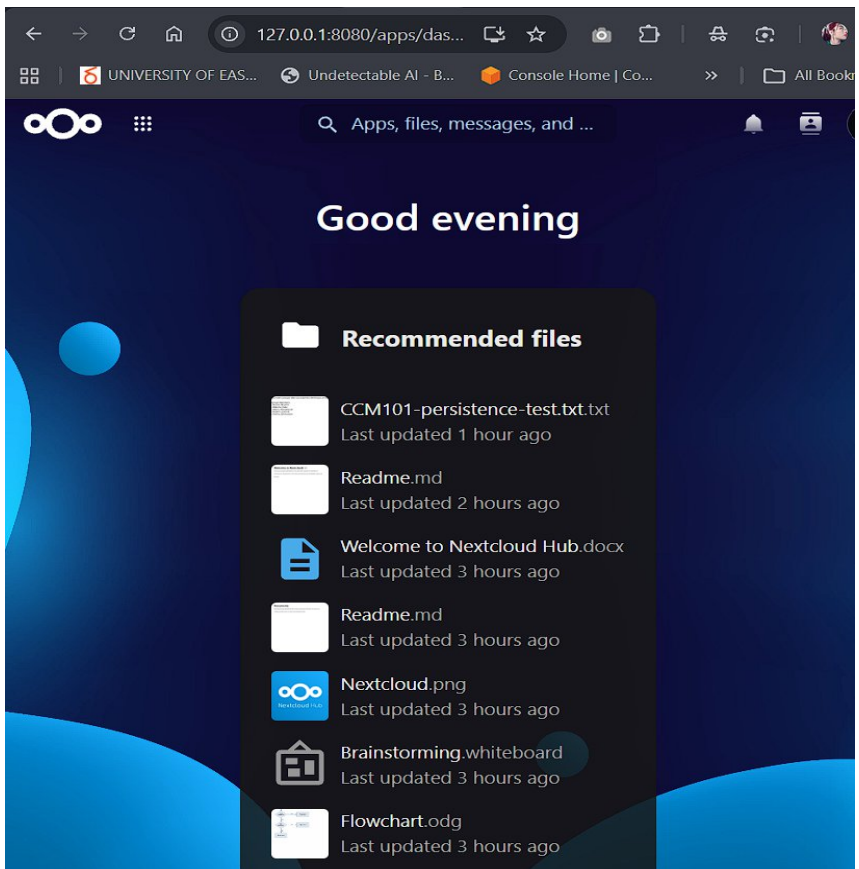
cloudadmin@vault-server:~/vaultcloud$ _

```

What it shows:

uptime -p shows the VM was freshly restarted, and docker compose ps shows **vault-app** running and **vault-db** running and **healthy** again, without any manual repair. This is the effect of the restart: unless-stopped policy and Docker starting at boot.

Figure 10. Test File Still Available After the Restart



What it shows:

The Nextcloud dashboard still lists **CCM101-persistence-test.txt**, so the data stored in the Docker volumes survived the restart.

Result: Persistent storage was verified: the data remained available after the VM was restarted.

6. SECURITY AND FIREWALL

6.1 Firewall Configuration (UFW)

UFW denies all incoming connections by default and allows outgoing connections. Only SSH and the Nextcloud web port are opened. The SSH rule uses limit, which is an allow rule that also blocks IP addresses making too many connection attempts.

Important: add the SSH rule **before** enabling the firewall, or the SSH session will be cut off.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw limit 22/tcp
sudo ufw allow 8080/tcp
sudo ufw enable
sudo ufw status verbose
```

```
cloudadmin@vault-server:~/vaultcloud$ sudo ufw limit 22/tcp comment SSH admin
ERROR: Need 'to' or 'from' clause
cloudadmin@vault-server:~/vaultcloud$
[ 1517.332107] watchdog: BUG: soft lockup - CPU#0 stuck for 42s! [bash:1839]
cloudadmin@vault-server:~/vaultcloud$ sudo ufw limit 22/tcp
Rules updated
Rules updated (v6)
cloudadmin@vault-server:~/vaultcloud$ sudo ufw allow 8080/tcp
Rules updated
Rules updated (v6)
cloudadmin@vault-server:~/vaultcloud$ sudo ufw enable
Firewall is active and enabled on system startup
cloudadmin@vault-server:~/vaultcloud$
cloudadmin@vault-server:~/vaultcloud$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22/tcp LIMIT IN Anywhere
8080/tcp ALLOW IN Anywhere
22/tcp (v6) LIMIT IN Anywhere (v6)
8080/tcp (v6) ALLOW IN Anywhere (v6)

cloudadmin@vault-server:~/vaultcloud$
```

Figure 11. UFW Firewall Status

What it shows:

`sudo ufw status verbose` shows **Status: active** and **Default: deny (incoming), allow (outgoing)**. The only rules are **22/tcp LIMIT IN** (SSH) and **8080/tcp ALLOW IN** (web), for both IPv4 and IPv6. (The first command in the screenshot failed because the comment text was not quoted; the rule was then added without a comment.)

6.2 Minimal Attack Surface

Control	Effect
Default deny incoming	Every port is closed unless it is explicitly allowed
Only 22 and 8080 allowed	Administration and the web application are the only services reachable
MariaDB publishes no port	The database cannot be reached from the VM network or the host
backend network is internal	Only the Nextcloud container can talk to the database
Secrets in .env (mode 600)	Passwords are not stored in the repository
Backups with mode 600	Only the owner can read the database dumps

Figure 12. Listening Ports and Database Isolation

```

cloudadmin@vault-server:/vaultcloud$ sudo ss -tulpn
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:* users:("systemd-resolve",pid=487,fd=18))
udp UNCONN 0 0 127.0.0.53:53 0.0.0.0:* users:("systemd-resolve",pid=487,fd=16))
udp UNCONN 0 0 10.0.2.15:dnsmasq:68 0.0.0.0:* users:("systemd-network",pid=666,fd=33))
udp UNCONN 0 0 127.0.0.1:323 0.0.0.0:* users:("chronyd",pid=869,fd=4))
udp UNCONN 0 0 [::]:323 [::]:* users:("chronyd",pid=869,fd=5))
tcp LISTEN 0 4096 127.0.0.53:53 0.0.0.0:* users:("systemd-resolve",pid=487,fd=17))
tcp LISTEN 0 4096 0.0.0.0:22 0.0.0.0:* users:("systemd",pid=1,fd=182))
tcp LISTEN 0 4096 127.0.0.54:53 0.0.0.0:* users:("systemd-resolve",pid=487,fd=19))
tcp LISTEN 0 4096 0.0.0.0:8080 0.0.0.0:* users:("docker-proxy",pid=1550,fd=8))
tcp LISTEN 0 4096 [::]:22 [::]:* users:("systemd",pid=1,fd=183))
tcp LISTEN 0 4096 [::]:8080 [::]:* users:("docker-proxy",pid=1555,fd=8))
cloudadmin@vault-server:/vaultcloud$ sudo docker port vault-db
cloudadmin@vault-server:/vaultcloud$

```

What it shows:

sudo ss -tulpn shows the TCP ports listening for services: **22** (SSH) and **8080** (Docker proxy for Nextcloud). Other entries are local system services (DNS resolver, DHCP client, time sync). sudo docker port vault-db prints nothing, which confirms that **MariaDB publishes no port**.

6.3 Known Limitation

Docker adds its own firewall rules for published ports, which can bypass UFW. For this reason the database port is never published, so its protection does not depend on UFW. Further hardening that could be added: HTTPS, SSH key-only login and filtering in the DOCKER-USER chain.

7. AUTOMATION AND SCHEDULED BACKUP

7.1 The Automation Script

automation-script.sh is a Bash script that checks the health of the platform and creates a verified backup of the Nextcloud database. It must be executable (chmod +x automation-script.sh). If any check fails it stops with an ERROR: message and exit code 1, and a half-written backup is never kept.

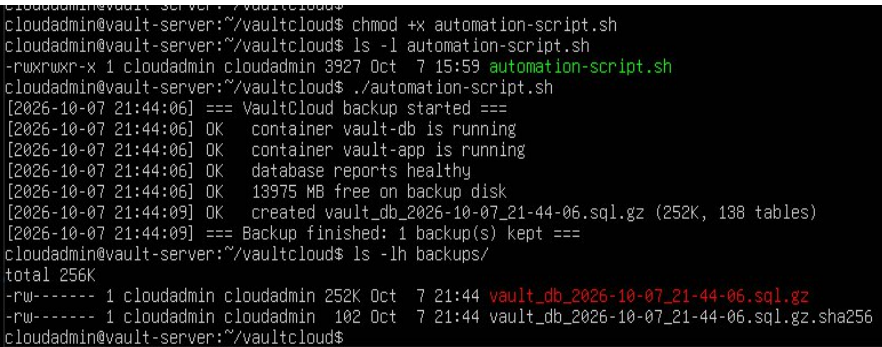
Step	What the script does
1	Takes a lock so two runs can never overlap
2	Checks that vault-db and vault-app are running and that the database reports healthy
3	Checks that at least 500 MB of disk space is free
4	Dumps the database inside the container (mariadb-dump) and compresses it to .sql.gz
5	Verifies the archive (gzip -t), checks that tables exist, and writes a SHA-256 checksum
6	Keeps the newest 7 backups and deletes older ones

Setting	Default	Meaning
BACKUP_DIR	backups/ next to the script	Where backups are stored
DB_CONTAINER / APP_CONTAINER	vault-db / vault-app	Containers to check and back up
KEEP	7	Number of backups to keep
MIN_FREE_MB	500	Minimum free disk space required

7.2 Run the Script Manually

```
cd ~/vaultcloud
chmod +x automation-script.sh
./automation-script.sh
ls -lh backups/
```

Figure 13. Automation Script Permissions and Execution



```
cloudadmin@vault-server:~/vaultcloud$
cloudadmin@vault-server:~/vaultcloud$ chmod +x automation-script.sh
cloudadmin@vault-server:~/vaultcloud$ ls -l automation-script.sh
-rwxrwxr-x 1 cloudadmin cloudadmin 3927 Oct  7 15:59 automation-script.sh
cloudadmin@vault-server:~/vaultcloud$ ./automation-script.sh
[2026-10-07 21:44:06] === VaultCloud backup started ===
[2026-10-07 21:44:06] OK   container vault-db is running
[2026-10-07 21:44:06] OK   container vault-app is running
[2026-10-07 21:44:06] OK   database reports healthy
[2026-10-07 21:44:06] OK   13975 MB free on backup disk
[2026-10-07 21:44:09] OK   created vault_db_2026-10-07_21-44-06.sql.gz (252K, 138 tables)
[2026-10-07 21:44:09] === Backup finished: 1 backup(s) kept ===
cloudadmin@vault-server:~/vaultcloud$ ls -lh backups/
total 256K
-rw----- 1 cloudadmin cloudadmin 252K Oct  7 21:44 vault_db_2026-10-07_21-44-06.sql.gz
-rw----- 1 cloudadmin cloudadmin 102 Oct  7 21:44 vault_db_2026-10-07_21-44-06.sql.gz.sha256
cloudadmin@vault-server:~/vaultcloud$
```

What it shows:

ls -l shows the script is executable (-rwxrwxr-x). Running it prints a timestamped log with both containers running, database healthy, about 13,975 MB free, and **vault_db_2026-10-07_21-44-06.sql.gz** created (252 KB, 138 tables). ls -lh backups shows the .sql.gz file and its .sha256 checksum, both with owner-only permissions (-rw-----).

7.3 Verify a Backup

```
cd ~/vaultcloud/backups
f=$(ls -1t vault_db_*.sql.gz | head -1)
sha256sum -c "$f.sha256"
gzip -t "$f" && echo "archive OK"
zcat "$f" | grep -c '^CREATE TABLE'
```

Figure 14. Backup Integrity Verification

```

root@cloudadmin:~# cd /home/cloudadmin/vaultcloud/automation-script.sh
cloudadmin@vault-server:~/vaultcloud$ cd ~/vaultcloud/backups
cloudadmin@vault-server:~/vaultcloud/backups$ f=$(ls -lt vault_db_*.sql.gz | head -1)
cloudadmin@vault-server:~/vaultcloud/backups$ sha256sum -c "$f.sha256"
vault_db_2026-10-07_21-44-06.sql.gz: OK
cloudadmin@vault-server:~/vaultcloud/backups$ gzip -t "$f" && echo "archive OK"
archive OK
cloudadmin@vault-server:~/vaultcloud/backups$ zcat "$f" | grep -c '^CREATE TABLE'
138
cloudadmin@vault-server:~/vaultcloud/backups$ cd ..
cloudadmin@vault-server:~/vaultcloud$ _

```

What it shows:

The checksum check prints **OK**, `gzip -t` prints **archive OK**, and the dump contains **138** `CREATE TABLE` statements, so the backup is a valid compressed archive of Nextcloud database.

7.4 Schedule the Backup with Cron

The script runs automatically every day at **3:15 AM** (VM local time). Open the crontab with `crontab -e` (choose nano the first time) and add this line at the bottom:

```

15 3 * * * /home/cloudadmin/vaultcloud/automation-script.sh >> /home/cloudadmin/vaultcloud/backups/cron.log 2>&1

```

The five fields mean minute 15, hour 3, every day of the month, every month, every day of the week. All output, including errors, is appended to `backups/cron.log`.

Figure 15. Scheduled Database Backup Using Cron

```

crontab: installing new crontab
cloudadmin@vault-server:~/vaultcloud$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * * tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
15 3 * * * /home/cloudadmin/vaultcloud/automation-script.sh >> /home/cloudadmin/vaultcloud/backups/cron.log 2>&1
cloudadmin@vault-server:~/vaultcloud$ tail backups/cron.log
tail: cannot open 'backups/cron.log' for reading: No such file or directory
cloudadmin@vault-server:~/vaultcloud$
cloudadmin@vault-server:~/vaultcloud$

```

What it shows:

crontab -l shows the installed job `15 3 * * *` pointing at `automation-script.sh` with output redirected to `backups/cron.log`. The log file is created by the first scheduled run, so it did not exist yet when `tail` was run in this screenshot.

Tip: To test the schedule without waiting, temporarily change the line to start with `* * * * *`, wait two or three minutes, check `backups/cron.log` and the new files in `backups/`, then change it back to `15 3 * * *`.

7.5 Restore Procedure (Disaster Recovery)

```

cd ~/vaultcloud
ls -lh backups/
zcat backups/vault_db_<date>.sql.gz | docker exec -i vault-
db sh -c \
'exec mariadb -uroot -p"$MARIADB_ROOT_PASSWORD"
"$MARIADB_DATABASE"'
docker compose restart app

```

Replace `<date>` with the timestamp of the backup to restore.
Verify the checksum first (Section 7.3).

8. OPERATIONAL PROCEDURES

Run all commands from the project folder ~/vaultcloud unless stated otherwise.

8.1 System Access

Access	How
Web application	Open <code>http://127.0.0.1:8080</code> in the Windows browser and sign in as the Nextcloud admin
Server administration	Windows Terminal: <code>ssh cloudadmin@127.0.0.1 -p 2222</code>
VM console	VirtualBox window (use only if SSH is unavailable)

8.2 Daily Operations

Task	Command
Check container status	<code>docker compose ps</code>
Start the system	<code>docker compose up -d</code>
Stop the system (data is kept)	<code>docker compose down</code>
Restart one service	<code>docker compose restart app</code>
View live logs	<code>docker compose logs -f app</code> (Ctrl+C to leave)
Nextcloud status	<code>docker exec -u www-data vault-app php occ status</code>
Run a manual backup	<code>./automation-script.sh</code>

Task	Command
List backups	ls -lh backups/
Firewall status	sudo ufw status verbose
List scheduled jobs	crontab -l
Disk space	df -h

Warning: Do **not** use `docker compose down -v`. It removes the volumes that hold all application and database data.

8.3 Update the Containers

1. Run a manual backup: `./automation-script.sh`.
2. Download new images: `docker compose pull`.
3. Re-create the containers: `docker compose up -d`.
4. Check `docker compose ps` and sign in to confirm the site works.

8.4 Change a Password or Setting

Edit `.env` with `nano .env`, then run `docker compose up -d` so the containers pick up the change. Database passwords are set when the database is first created, so changing them later also requires changing them inside MariaDB.

8.5 Shut Down and Start the VM Safely

1. Shut down from inside the VM with `sudo shutdown now` (do not power off the laptop while the VM runs).
 2. Start the VM again from VirtualBox. The containers start automatically.
 3. Wait about 30 seconds for the database to become healthy, then check `docker compose ps`.
-

9. MAINTENANCE AND TROUBLESHOOTING

9.1 Maintenance Schedule

When	Task
Every day (automatic)	Cron runs the health check and database backup at 3:15 AM
Every week	Read backups/cron.log for ERROR lines; confirm 7 backups exist with <code>ls -lh backups/</code> ; run <code>docker compose ps</code>
Every month	Run <code>sudo apt update && sudo apt upgrade</code> , reboot, and confirm both containers return; check <code>df -h</code> and <code>sudo ufw status</code>
Before any change	Run a manual backup first
Every term	Test a restore from a backup so recovery is proven, not assumed

9.2 Troubleshooting

Symptom	Cause and fix
Symbols such as <code> </code> , <code><</code> or <code>></code> type wrongly in the VM console	Console keyboard layout does not match the physical keyboard. Use SSH from Windows Terminal, or run <code>sudo dpkg-reconfigure keyboard-configuration</code>
VM freezes or the laptop shuts down; soft lockup messages	The host is short of RAM or CPU. Close other programs, keep the VM at about 2 GB, plug in power, and shut the VM down cleanly
After a restart the site is down	Wait for the database to be healthy. If containers are not running: <code>docker compose up -d</code>

Symptom	Cause and fix
Browser says "Access through untrusted domain"	Run <code>docker exec -u www-data vault-app php occ config:system:set trusted_domains 2 --value=<host></code>
permission denied when running docker	Log out and in again after <code>sudo usermod -aG docker \$USER</code>
ufw reports "Need 'to' or 'from' clause"	A comment was not quoted. Quote it or add the rule without a comment
Script: Permission denied	<code>chmod +x automation-script.sh</code>
Script: bad interpreter or ^M	Windows line endings: <code>sed -i 's/\r\$//' automation-script.sh</code>
No backup appears at 3:15 AM	Check <code>systemctl status cron</code> , <code>crontab -l</code> and <code>backups/cron.log</code> ; use absolute paths in cron
SSH locked out after enabling UFW	Use the VirtualBox console and run <code>sudo ufw allow 22/tcp</code>
REMOTE HOST IDENTIFICATION HAS CHANGED on Windows	The VM was reinstalled. Run <code>ssh-keygen -R "[127.0.0.1]:2222"</code> and connect again

10. SOURCE CONTROL (GITHUB)

All code and documentation are committed to the repository folder below. The real .env file and the backups/ folder are intentionally **not** committed.

Repository: <https://github.com/katkit-del/CCM101-katemalona>

```
Laboratory-10-Enterprise-Cloud-Architect/
├── README.md
├── architecture-diagram.png
├── docker-compose.yml
├── .env.example
├── .gitignore
├── automation-script.sh
├── operational-manual.md
├── final-reflection.md
└── screenshots/
```

10.1 Publish Changes

```
cd ~/CCM101-katemalona
git status          # .env and backups/ must NOT be
                    listed
git add Laboratory-10-Enterprise-Cloud-Architect
git commit -m "Update Laboratory 10"
git push
```

10.2 Redeploy on a Fresh Environment (for example KillerCoda)

```
docker --version | | curl -fsSL https://get.docker.com | sh
git clone https://github.com/katkit-del/CCM101-katemalona.git
cd CCM101-katemalona/Laboratory-10-Enterprise-Cloud-Architect
cp .env.example .env && nano .env
```

```
docker compose up -d  
chmod +x automation-script.sh && ./automation-script.sh
```

Open the environment's **Traffic / Ports** menu for port 8080. If the site reports an untrusted domain, add that hostname with the `occ` command in Section 9.2.

11. FINAL REFLECTION

11.1 What We Learned

This project showed how the separate skills from the earlier missions work together in one enterprise-style environment: a Linux server in a virtual machine, NAT networking and port forwarding, containers described as code with Docker Compose, a default-deny firewall, persistent storage, and automated, verified backups. Building it end to end also showed that deploying a service is only half the job. It must also be secured, monitored, backed up and documented so that someone else can run it.

11.2 Our Journey: Mission 1 to Mission 10

Mission	Title	What we actually did
Mission 1	Welcome to the Cloud	We set up our GitHub portfolio, wrote our first system-information.md, and got used to working inside a Linux environment. This was the first time most of us used the terminal for anything serious.
Mission 2	Linux Fundamentals	We practiced file navigation, permissions, and basic commands like chmod, chown, and grep. Most of our early mistakes were typos in paths — a habit that still bit us during this final project.
Mission 3	Multi-Cloud Explorer	We compared AWS, Azure, and GCP through their free tiers, took screenshots of each console, and wrote research notes on pricing and use cases. This is where we first understood what "cloud" actually meant beyond the marketing slides.
Mission 4	Version Control	We learned branching, merging, and how to push to a remote repository without breaking

Mission	Title	What we actually did
	with Git	anything. We also learned to actually read git status before committing — something that saved us during Lab 10.
Mission 5	Containerization Basics	Our first Docker container. We pulled hello-world, ran it, and then built a small custom image with a Dockerfile. Seeing a program run identically on every laptop was the moment containers clicked for us.
Mission 6	Docker Networking and Volumes	We connected two containers on the same network and mounted a named volume. This was the direct foundation for docker-compose.yml in Lab 10 — the frontend/backend split and the db_data volume are things we already understood by then.
Mission 7	Infrastructure as Code with Docker Compose	We wrote our first multi-service docker-compose.yml file. This mission taught us that YAML indentation is unforgiving and that docker compose config -q is worth running before every up -d.
Mission 8	Cloud Networking and Firewalls	We configured UFW, set up port forwarding, and learned the hard way that enabling a firewall without allowing SSH first locks you out. We used that exact rule ordering in Section 6 of this manual.
Mission 9	Monitoring and Automation	We wrote our first Bash script, scheduled it with Cron, and learned to log output instead of just printing it to the terminal. Our automation-script.sh in Lab 10 is a direct

Mission	Title	What we actually did
	on	descendant of this mission's script.
Mission 10	The Enterprise Cloud Architect	This is the one you are reading right now. We designed, deployed, secured, automated, and documented a complete Nextcloud + MariaDB stack on Ubuntu Server — with no step-by-step guide. It pulled every earlier mission together, and it is the first time we built something that felt like real production work.

11.3 Challenges and Solutions

1. **Keyboard layout in the VM console.** Symbols such as `|`, `<` and `>` were typed as other characters in the VirtualBox console. We solved it by working over SSH from Windows Terminal, which uses the normal Windows keyboard and also allows copy and paste.
 2. **VM freezes and an unexpected laptop shutdown.** The laptop has limited RAM, and the VM showed soft lockup kernel warnings before the laptop shut down and took the VM with it. We kept the VM at about 2 GB, closed other programs, restarted the VM and confirmed that both containers came back by themselves and that the data was intact. This confirmed the value of `restart: unless-stopped` and of persistent volumes.
 3. **Firewall rule error.** The first `ufw limit` command failed with "Need 'to' or 'from' clause" because the comment was not quoted. We added the rule without the comment and verified the result with `ufw status verbose`.
-

11.4 Project Outcome

We delivered a working Nextcloud and MariaDB platform on Docker Compose with persistent storage, a default-deny UFW firewall, an internal-only database network and a scheduled, verified daily database backup. The screenshots in this manual show each of these results.

11.5 Future Improvements

- HTTPS with a valid certificate in front of Nextcloud.
- Copy backups off the VM (another disk, server or cloud storage).
- SSH key-only login and fail2ban.
- Monitoring and alerts for failed backups and unhealthy containers.
- Restrict Docker-published ports with rules in the DOCKER-USER chain.

11.6 AI Assistance Disclosure

An AI assistant was used as a guide and reference throughout this project. It helped us understand concepts we were unfamiliar with, suggested possible approaches when we got stuck, and provided sample command structures that we then reviewed, tested, and adapted to fit our own setup. It also served as a troubleshooting partner when we encountered issues like the VirtualBox keyboard layout problem and the VM freezes.

All configuration files, commands, screenshots, and documentation in this manual were prepared, executed, and verified by the group ourselves. The AI's role was to assist our learning process — not to replace it. Every step was performed on our own machines, and every result shown in this manual is from our actual VaultCloud deployment.